

Lab Companion — Create and Execute a SQL Workflow in Dataform

The anchor lab of Day 1 (Mon Oct 5, 3:25–4:05 PM CT): you build a working Dataform SQL workflow yourself, right after the demo repo walkthrough. The lab's step-by-step instructions live in the Google Skills classroom — this companion orients you, flags what commonly goes wrong, and connects the lab to your migration. It does not replace the lab instructions.

What you'll do

The lab (**Create and Execute a SQL Workflow in Dataform**, Course 1, Module 4, in the [Skills classroom](#)) has four objectives:

1. Create a Dataform repository.
2. Create and initialize a Dataform development workspace.
3. Create and execute a SQL workflow.
4. View execution logs in Dataform.

Why it matters for your migration

Your target architecture names Dataform as the Silver→Gold ELT tool. Everything else today is context; these 40 minutes are the thing you will actually do at work. You have just watched a real raw → staging → marts repository end to end (see the [repo tour](#)); now you build the same bones — repo, workspace, sqlx, execute — with your own hands, in your own lab project.

Before you start

- Launch the lab from the [Skills classroom](#); it provisions your GCP project and credentials. Work in that lab-provided project, not any other you may see.
- **Check the project picker** at the top of the console every time you sign in — the console loves to default to the wrong project. If lab resources seem "missing," look here first.
- Dataform lives inside the BigQuery console (BigQuery Studio → Dataform). Console navigation shifts occasionally; if the menu has moved, search "Dataform" in the console search bar.
- If the console prompts you to **enable an API** (BigQuery, Dataform), click Enable — that is normal on a fresh project.
- When the lab asks for a **region**, use the one the lab specifies and use it consistently. A repository in one region and datasets in another surfaces later as empty executions and confusing errors.

What you'll build

The arc, so the clicks make sense:

1. A **staging view** over the source data — Silver-layer cleaning logic.
2. A **mart table** that references it with `ref()` — a Gold-layer aggregate.
3. Dataform **compiles the dependency graph** from your `ref()` calls — watch for it.
4. You **execute the workflow** and read the logs — Dataform runs the actions in dependency order and records each one.

Staging view → mart table → graph → run. The same skeleton as the walkthrough repo, smaller.

While you work — pointers

- **The compiled graph** is a tab inside your development workspace. It lags a beat after you edit a `sqlx` file — if a new model does not appear, wait a few seconds before assuming the file did not save.
- **Executing is not committing.** Running the workflow builds tables and views in BigQuery. Committing versions the code in git. Neither one is a deployment — at work, deployment is a release configuration running off the repo (the 4:05 PM automation block shows the trigger).
- **Execution logs** show each action's status, duration, and the exact compiled SQL BigQuery ran. When something turns red, that log is the first place to look — same instinct as opening the SQL Agent job history.

For the CI/CD-minded

This lab tends to trigger the sharpest questions of the day from teams that already run deployment pipelines. Short answers, so you can keep building:

- **'Is a workspace a branch?'** Effectively, yes: an isolated development copy of the repo, backed by git — your feature branch. You commit and push changes; releases run off the repository, not off your workspace.
- **'Where is the git remote?'** The lab's repository starts as local git. The walkthrough repo is connected to GitHub — which is exactly what you would do at work.
- **'Can two people edit at once?'** Separate development workspaces, merged through git — the same answer as your current shop.
- **'How does this get to prod?'** Release configurations (a git commit-ish compiled into a result) plus scheduled workflow invocations. Your deployment pipeline, translated — Dataform slots into the discipline you already have; it does not ask you to abandon it.

Common gotchas

- **Workspace looks empty or 'broken':** you missed the **Initialize workspace** button. This is the number-one stall in this lab — the workspace shows no files until you initialize it.
- **Region mismatch** at repository creation → executions appear to run but results look empty or land somewhere unexpected. Fix: recreate the repository in the correct region.

- **IAM error on execution:** executions run as a service account; if it lacks BigQuery permissions the run fails with a permissions error. Flag the instructor rather than burning ten minutes on it.
- **'My edit didn't take':** compiled-graph lag. Confirm the file saved, give the graph a few seconds, then reload.
- **Everything looks missing:** the project picker. It is always the project picker.

If you finish early

Expect to — SQL people fly through this lab. In rough order of value:

1. **Add an assertion** to one model — a uniqueKey or a non-null rowConditions check — and re-run. Watch it appear as a node in the compiled graph and as a result in the execution logs. (A few lines of config; the [repo tour](#) shows the exact shape.)
2. **Flip a model to type: 'incremental'** and reason about when you would use it — think of your largest source table from this morning's sizing discussion.
3. **Interrogate the compiled graph:** find the mart you just made. What depends on it? What does it depend on? That answer is what your code review will look like at work.

Self-check

- ☐ Dataform repository created — in the lab project, in the region the lab specified.
- ☐ Development workspace created **and initialized** (files visible).
- ☐ Staging view and mart table defined in sqlx, wired together with ref().
- ☐ Workflow executed successfully — all actions green.
- ☐ Execution logs opened and read, including the compiled SQL.
- ☐ Compiled graph located and explored.
- ☐ Stretch: assertion added and observed in a run.

What this replaces in your current stack

The hand-sequenced transform packages, the "run these three stored procedures in order" SQL Agent steps, and the morning-after manual validation queries. You keep: SQL, git, code review, scheduled deployments. You lose: the designer surface, the shared scheduler box, and validation that lives outside the pipeline.

Debrief, in one sentence: you just did ELT the way your target architecture will do it — SQL models in git, compiled into a dependency graph, executed in order, tested where the data is made.

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.